

Reviewer Pack — Symmetric Group Irreducible Component Exponential Gap in Per...

Entry 275156ccf299 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 19:24:43 UTC

Symmetric Group Irreducible Component Exponential Gap in Permanent vs Determinant

Entry ID: 275156ccf299 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 19:24:43 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Monotone Circuit Size

Statement:

For $n \geq 4$, the number of distinct irreducible representations in the decomposition of the permanent polynomial under S_n is $\Omega(2^n)$, while the determinant decomposes into exactly one irreducible representation. This exponential gap persists under linear substitutions.

Rationale (proposer's reasoning):

The symmetric group's representation theory reveals structural asymmetry between permanent and determinant. The permanent's decomposition into irreducible components reflects its inherent complexity, while the determinant's single-component structure hints at algebraic simplicity. This asymmetry could translate to computational hardness in monotone circuits.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2391dd588d57394c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def binomial_coefficient(n, k):
    return factorial(n) // (factorial(k) * factorial(n - k))

def hook_length_formula(shape):
    n = len(shape)
    total = 0
    for row in range(n):
        for col in range(len(shape[row])):
            total += shape[row][col] + 1 - row - col
    return binomial_coefficient(total, n)

def generate_irreducible_components(n):
    if n == 2:
        return [1]
    components = []
    for i in range(1, n):
        components.extend(generate_irreducible_components(i))
    components.append(n)
```

```

    return components

def count_irreducible_representations(shape):
    components = generate_irreducible_components(len(shape))
    return len(components)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(4, 40)
    permanent_shape = [[n - i - j for j in range(i + 1)] for i in range(n)]
    determinant_shape = [[1] * (i + 1) for i in range(n)]

    permanent_components = count_irreducible_representations(permanent_shape)
    determinant_components = count_irreducible_representations(determinant_shape)

    metric_name = "irreducible_components"
    metric_value = permanent_components - determinant_components
    instances_tested = 1
    conjecture_holds = metric_value >= 2**(n // 2)
    counterexample = "" if conjecture_holds else f"Permanent: {permanent_components}, Determinant: {determinant_components}"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 100))[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Permanent exceeds determinant by less than 2^{first_failing_seed}\"")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing validation of the conjecture’s support fraction or counterexample detection. | next: Increase timeout duration and re-run the test with stricter resource limits to ensure completion.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111488
2	propose	ollama_remote	qwen3:8b	0	0	110102
3	propose	ollama_remote	qwen3:8b	0	0	82054
4	preregistration	ollama_remote	qwen3:8b	0	0	24066
5	novelty	ollama_remote	qwen3:8b	0	0	20805
6	novelty	ollama_remote	qwen3:8b	0	0	14676
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13271
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9043
9	judge	ollama_remote	qwen3:8b	0	0	91868

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 477375 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/275156ccf299.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/275156ccf299.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/275156ccf299.tar.gz* (if generated)